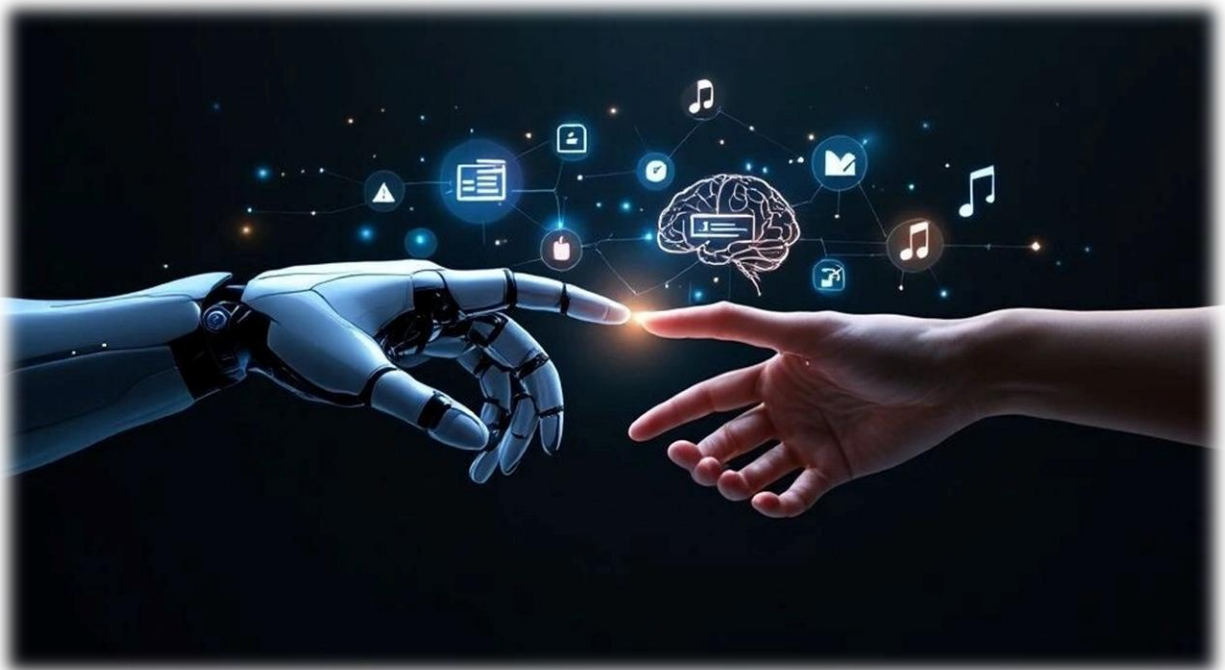




AIR UNIVERSITY, ISLAMABAD



GENERATIVE AI FOR SOFTWARE ENGINEERING

SUBMITTED TO: DR HUMAIRA WAQAS

SUBMITTED BY: ALIZA ZAMAN | 232016-BSSE-6B | 2026

LAB 07**Activity 1:****Build a FastAPI server that implements the Reflection Pattern for Python code review**

- *Health check*

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /health
- Parameters:** No parameters
- Execute:** Button to execute the request
- Clear:** Button to clear the request
- Responses:** Section containing:
 - Curl:** `curl -X 'GET' \ 'http://127.0.0.1:8000/health' \ -H 'accept: application/json'`
 - Request URL:** `http://127.0.0.1:8000/health`
 - Server response:**
 - Code:** 200
 - Response body:** `{ "status": "ok", "model": "Llama-3.1-8b-Instant" }`
 - Response headers:** `content-length: 46 content-type: application/json date: Fri, 24 Apr 2026 05:12:33 GMT server: uvicorn`
- Responses Table:**

Code	Description	Links
200	Successful Response	No links

Lab 07

- **Test 1 Simple Task, Quick Approval:**

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/reflect' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "task": "Write a Python function that checks if a string is a palindrome.",
    "max_rounds": 3
  }'
```

Request URL

http://127.0.0.1:8000/reflect

Server response

Code Details

200

Response body

```
{
  "final_code": "def is_palindrome(input_string: str) -> bool:\n    \"\"\"Checks if a given string is a palindrome, ignoring non-alphanumeric characters and case.\"\"\" Args:\n    input_string (str): The input string to check.\n    Returns:\n    bool: True if the string is a palindrome, False otherwise.\n    Raises:\n    ValueError: If the input string is empty, contains only whitespace characters, or contains non-ASCII characters.\n    \"\"\"\n    if not isinstance(input_string, str) or input_string is None:\n        raise TypeError(\"Input must be a non-None string\")\n    if not input_string.strip():\n        raise ValueError(\"Input string is empty\")\n    cleaned_string = ''.join(c for c in input_string if c.isalnum()).casefold()\n    if not cleaned_string.strip():\n        raise ValueError(\"Input string is empty after removing whitespace\")\n    if not cleaned_string.isascii():\n        raise ValueError(\"Input string contains non-ASCII characters\")\n    left, right = 0, len(cleaned_string) - 1\n    while left < right:\n        if cleaned_string[left] != cleaned_string[right]:\n            return False\n        left += 1\n        right -= 1\n    return True",
  "round_count": 3,
  "approved": false,
  "critiques": [
    "ISSUE [Correctness]: The function does not handle non-string inputs. If a non-string input is passed to the function, it will raise an error when trying to call the 'isalnum()' method on it.\nISSUE [Readability]: The variable name 's' is not descriptive. It would be better to use a more descriptive name, such as 'input_string'.\nISSUE [Efficiency]: The function creates a new string when calling 's[::-1]', which involves creating a new list and then joining it into a string. This is not the most efficient way to reverse a string in Python, especially for large strings. A more efficient way would be to use a two-pointer approach or the 'reversed()' function.\nISSUE [Security]: The function does not check if the input is 'None'. If 'None' is passed to the function, it will raise an error when trying to call the 'isalnum()' method on it.\nISSUE [Edge cases]: The function does not handle empty strings. While it is technically correct to return 'True' for an empty string, it might be more intuitive to raise a 'ValueError' in this case, as an empty string is not a palindrome in the classical sense.",
    "ISSUE [Correctness]: The function does not handle non-ASCII characters correctly. The 'isalnum()' method only considers ASCII alphanumeric characters, and the 'lower()' method only considers ASCII lowercase letters. This means that the function may incorrectly identify a string as a palindrome if it contains non-ASCII characters.\nISSUE [Readability]: The docstring could be more descriptive. It does not explain why the function raises a 'ValueError' for an empty string, or why it raises a 'TypeError' for a non-string input. It also does not mention the cleaning process that is performed on the input string.\nISSUE [Efficiency]: The function uses the 'reversed()' function to reverse the cleaned string, which has a time complexity of O(n). This is not necessary, as we can simply compare the cleaned string with its reverse character by character, which has a time complexity of O(n/2).\nISSUE [Security]: The function does not validate the input string for any security concerns. However, since it does not involve any user input or external data, this is not a major concern.\nISSUE [Edge cases]: The function does not handle the case where the input string is a single character. While this is technically a palindrome, the function will still return 'True' for this case. However, this is not a major issue.",
    "ISSUE [Correctness]: The function does not handle non-string inputs correctly. It raises a 'TypeError' for non-string inputs, but it does not handle other types of inputs that might cause errors. For example, it should also raise a 'TypeError' for inputs like 123 or [1, 2, 3].\nISSUE [Readability]: The function raises a 'ValueError' for an empty string, but it does not provide an explanation for this error."
  ]
}
```

Response headers

```
content-length: 6822
content-type: application/json
date: Fri, 24 Apr 2026 05:13:12 GMT
server: uvicorn
```

- **Test 2 Complex Task, Full 3 Rounds:**

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/reflect' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "task": "Write a Python function that takes a list of integers and returns the two indices whose values sum to a given target. Raise a ValueError if no solution exists.",
    "max_rounds": 3
  }'
```

Request URL

http://127.0.0.1:8000/reflect

Server response

Code Details

200

Response body

```
{
  "final_code": "def two_sum(nums, target):\n    \"\"\"Returns all pairs of indices of two numbers in the list that add up to the target value.\"\"\" Args:\n    nums (list): A list of integers.\n    target (int): The target sum.\n    Returns:\n    list: A list of tuples, where each tuple contains the indices of two numbers that add up to the target.\n    Raises:\n    ValueError: If the input list is empty, contains non-integer values, or the target value is not an integer.\n    TypeError: If the input list or target value is not of the expected type.\n    \"\"\"\n    if not isinstance(nums, list) or not all(isinstance(num, int) for num in nums):\n        raise TypeError(\"Input list must be a list of integers\")\n    if not isinstance(target, int):\n        raise ValueError(\"Target value must be an integer\")\n    if len(nums) == 1:\n        raise ValueError(\"Input list contains only one element\")\n    num_indices = {}\n    two_sum_pairs = []\n    for i, num in enumerate(nums):\n        target_complement = target - num\n        if target_complement in num_indices:\n            two_sum_pairs.append((num_indices[target_complement], i))\n        num_indices[num] = i\n    if not two_sum_pairs:\n        raise ValueError(\"No solution exists\")\n    return two_sum_pairs",
  "round_count": 3,
  "approved": false,
  "critiques": [
    "ISSUE [Correctness]: The function does not handle the case when the input list 'nums' is empty. In this case, the function should raise a 'ValueError' or return a specific value to indicate that no solution exists.\nISSUE [Readability]: The variable name 'complement' could be more descriptive. Consider renaming it to 'target_complement' to make it clear what it represents.\nISSUE [Edge cases]: The function does not handle the case when the input list contains duplicate numbers. It will still return the correct indices, but it's worth noting that the problem statement does not explicitly state that the input list should not contain duplicates.\nISSUE [Efficiency]: The function has a time complexity of O(n), which is optimal for this problem. However, the space complexity is also O(n) due to the use of a dictionary to store the numbers and their indices. While this is acceptable for most use cases, it's worth noting that the function uses more memory than necessary.",
    "ISSUE 1: Correctness - The function does not handle the case when there are multiple pairs of numbers that add up to the target value. It will return the first pair it finds. If there are multiple pairs, it will only return one of them.\nISSUE 2: Edge cases - The function does not handle the case when the input list contains duplicate numbers. It will still return the correct indices, but it's worth noting that the problem statement does not explicitly state that the input list should not contain duplicates.\nISSUE 3: Readability - The variable name 'target_complement' could be more descriptive. Consider renaming it to something like 'target_complement' to better convey its purpose.\nISSUE [Readability]: The variable name 'num_dict' could be more descriptive. Consider renaming it to something like 'num_indices' to better convey its purpose.",
    "ISSUE [Correctness]: The function does not handle the case where the input list contains duplicate numbers. Although it does not affect the correctness of the two-sum problem, it is a desirable feature to handle duplicates in a more robust way.\nISSUE [Readability]: The variable name 'num_dict' could be more descriptive. Consider renaming it to something like 'num_indices' to better convey its purpose."
  ]
}
```

Response headers

```
content-length: 6370
content-type: application/json
date: Fri, 24 Apr 2026 05:20:58 GMT
server: uvicorn
```

Responses

Lab 07

- **Test 3 Force Early Stop (max_rounds=1):**

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/reflect' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "task": "Write a Python function that checks if a string is a palindrome.",
    "max_rounds": 1
  }'
```

Request URL

http://127.0.0.1:8000/reflect

Server response

Code

Details

200

Response body

```
{
  "final_code": "def is_palindrome(s: str) -> bool:\n    \"\"\"\n    Checks if a given string is a palindrome.\n    Args:\n        s (str): The input string to check.\n    Returns:\n        bool: True if the string is a palindrome, False otherwise.\n    Notes:\n        This function is not case sensitive and ignores spaces and punctuation.\n        \"\"\"\n    if not isinstance(s, str) or len(s) == 0:\n        raise ValueError(\"Input must be a non-empty string\")\n    s = ''.join(c for c in s if c.isalnum()).lower()\n    left, right = 0, len(s) - 1\n    while left < right:\n        if s[left] != s[right]:\n            return False\n        left += 1\n        right -= 1\n    return True",
  "round_count": 1,
  "approved": false,
  "critiques": [
    "ISSUE [Correctness]: The function is case sensitive and considers spaces and punctuation as part of the string, which may not be the desired behavior for all users. This could lead to incor",
    "rect results for inputs like 'A man, a plan, a canal: Panama'.\n\nISSUE [Readability]: The function does not handle edge cases like an empty string or None input. It would be better to add inp",
    "ut validation to handle these cases.\n\nISSUE [Efficiency]: The function uses slicing to reverse the string, which creates a new string in memory. This could be inefficient for large strings. A",
    "more efficient approach would be to use a two-pointer technique to compare characters from the start and end of the string.\n\nISSUE [Security]: The function does not handle user input, but it d",
    "oes not pose a security risk in this case.\n\nISSUE [Edge cases]: The function does not handle edge cases like None or empty string input. It would be better to add input validation to handle th",
    "ese cases.\n\nISSUE [Readability]: The function could benefit from additional comments or docstrings to explain the two-pointer technique used in the alternative implementation.\n\nWhere's an up",
    "dated version of the function that addresses these issues:\n\n\"\"\"\npython\ndef is_palindrome(s: str) -> bool:\n    \"\"\"\n    Checks if a given string is a palindrome.\n    Args:\n        s (st",
    "r): The input string to check.\n    Returns:\n        bool: True if the string is a palindrome, False otherwise.\n    Notes:\n        This function is case sensitive and considers spaces and",
    "punctuation as part of the string.\n    \"\"\"\n    if not isinstance(s, str):\n        raise ValueError(\"Input must be a string\")\n    s = ''.join(c for c in s if c.isalnum()).lower()\n    le",
    "ft, right = 0, len(s) - 1\n    while left < right:\n        if s[left] != s[right]:\n            return False\n        left += 1\n        right -= 1\n    return True\n\"\"\""
```

Response headers

```
content-length: 2658
content-type: application/json
date: Fri, 24 Apr 2026 05:22:01 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Page 4 | 11

Lab Task 1:**Swap Groq for Ollama (Local, Free)**

- *Health check:*

Reflection Agent - Ollama 1.0 OAS 3.1

/openapi.json

default

GET /health Health

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/health' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8001/health

Server response

Code	Details
200	Response body <pre>{ "status": "ok", "model": "qwen2.5:0.5b" }</pre> Response headers <pre>content-length: 38 content-type: application/json</pre>

- *Test 1 Palindrome Task with Valid Function & Critic Array:*

Ollama Response(8001)

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/reflect' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "task": "Write a Python function that checks if a string is a palindrome.",
    "max_rounds": 3
  }'
```

Request URL

http://127.0.0.1:8001/reflect

Server response

Code	Details
200	Response body <pre>{ "final_code": "python\ndef is_palindrome(s: str) -> bool:\n \"\"\" Checks if the given string is a palindrome.\n\n A palindrome is a word, phrase, or sequence that reads the same\n backward as forward.\n\n for example, 'radar' and 'madam' are palindromes. However, 'hello' is not a palindrome.\n\n Args:\n s (str): The input string to check.\n\n Return\n bool: True if the input string is a palindrome, False otherwise.\n\n \"\"\"\n # Handle empty strings by converting them to an empty list\n cleaned_s = [char for char in s]\n # Compare characters from both ends moving towards the center\n for i in range(len(cleaned_s) // 2):\n if cleaned_s[i] != cleaned_s[-i - 1]:\n return False\n return True\n\n# Test cases to verify the correctness of the function\nassert is_palindrome('radar') == True, 'Test case 1 failed'\nassert is_palindrome('madam') == True, 'Test case 2 failed'\nassert is_palindrome('hello') == False, 'Test case 3 failed'\n\nprint('All test cases passed!')"</pre> Response headers <pre>content-length: 1593 content-type: application/json date: Fri, 24 Apr 2026 05:31:32 GMT server: uvicorn</pre>

Responses

Code	Description
------	-------------

Links

Groq Response (8000)

```
curl -X 'POST' \
  'http://127.0.0.1:8000/reflect' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "task": "Write a Python function that checks if a string is a palindrome",
    "max_rounds": 3
  }'.
```

Request URL
http://127.0.0.1:8000/reflect

Server response

Code Details

200

Response body

```
{
  "final_code": "def is_palindrome(input_string: str) -> bool:\n    \"\"\"Checks if a given string is a palindrome.\"\"\"\n    Args:\n        input_string (str): The input string to check.\n    Returns:\n        bool: True if the string is a palindrome, False otherwise.\n    Raises:\n        ValueError: If the input is not a string.\n    \"\"\"\n    if not isinstance(input_string, str):\n        raise ValueError(\"Input cannot be None\")\n    if not input_string: # Check for an empty string\n        return False\n    alphanumeric_string = ''.join(char for char in input_string if char.isalnum()).lower()\n    start = 0\n    end = len(alphanumeric_string) - 1\n    while start < end:\n        if alphanumeric_string[start] != alphanumeric_string[end]:\n            return False\n        start += 1\n        end -= 1\n    return True",
  "round_count": 3,
  "approved": false,
  "critiques": [
    "ISSUE [Correctness]: The function does not handle None input. If None is passed to the function, it will raise an AttributeError when trying to call the isalnum() method on it.\nISSUE [Readability]: The function name is_palindrome is clear, but the variable name 's' could be more descriptive. Consider renaming it to something like input_string.\nISSUE [Readability]: The Notes section in the docstring mentions that the function is case-sensitive and considers spaces and punctuation as part of the string, but this is not immediately clear from the function name or the docstring. Consider adding this information to the docstring.\nISSUE [Efficiency]: The function creates a new string by joining the alphanumeric characters and converting to lower case. This creates a new string object, which can be inefficient for large strings. Consider using a generator expression to yield the characters instead of creating a new string.\nISSUE [Security]: The function does not handle user input. However, since it does not use eval() or any other function that could pose a security risk, this is not a major concern. Nevertheless, it's always a good practice to validate user input.\nHere's the corrected code:\npython\ndef is_palindrome(input_string: str) -> bool:\n    \"\"\"Checks if a given string is a palindrome.\"\"\"\n    Args:\n        input_string (str): The input string to check.\n    Returns:\n        bool: True if the string is a palindrome, False otherwise.\n    Notes:\n        This function is case-sensitive and considers spaces and punctuation as part of the string.\n    \"\"\"\n    if input_string is None:\n        raise ValueError(\"Input cannot be None\")\n    cleaned_string = ''.join(c for c in input_string if c.isalnum()).lower()\n    return cleaned_string == cleaned_string[::-1]",
    "ISSUE [Correctness]: The function raises a ValueError for None input, but the docstring does not mention this behavior. It says the function is case-sensitive and considers spaces and punctuation as part of the string, but it actually removes them.\nISSUE [Readability]: The variable name 'c' is not very descriptive. It would be better to use a name like 'char' or 'character'. \nISSUE [Readability]: The variable name 'cleaned_string' is not very descriptive. It would be better to use a name like 'alphanumeric_string' or 'cleaned_input'. \nISSUE [Efficiency]: The function creates a new string with the cleaned characters, and then checks if it's equal to the reversed string. This is not the most efficient way to check if a string is a palindrome, especially for large strings. It would be more efficient to compare characters from the start and end of the string, working towards the center.\nISSUE [Security]: Although the function does not use eval on user input, it does not validate the input type. It should check if the input is indeed a string.\nISSUE [Readability]: The variable names 'left' and 'right' are not descriptive. They should be renamed to something like 'start' and 'end' to clearly indicate their purpose.\nISSUE [Efficiency]: The code checks if the input string is None before checking if it's a string. It would be more efficient to check the type first, as 'isinstance(None, str)' is False.\nISSUE [Correctness]: The code checks if the input string is None before checking if it's a string. It would be more efficient to check the type first, as 'isinstance(None, str)' is False.\nISSUE [Correctness]: The code checks if the input string is None before checking if it's a string. It would be more efficient to check the type first, as 'isinstance(None, str)' is False."
  ]
}
```

Response headers

```
content-length: 5428
content-type: application/json
date: Sat, 09 May 2026 11:57:32 GMT
server: uvicorn
```

Responses

Reflection:

This swap made clear to me that the Reflection Pattern is an architectural concept that exists independently of whatever LLM or platform is executing it. The loop structure, the Generator and Critic roles, the APPROVED stopping condition, and the response schema are all design decisions, they live in the code I wrote, not inside Groq or Ollama. Changing the provider required modifying exactly one function, which shows that the pattern treats the LLM as an interchangeable component rather than a core part of the architecture. This separation is actually really important in production systems because it means you can switch models for cost, speed, or capability reasons without touching any of the agent logic. It also means the same pattern can be evaluated across different models to compare output quality, which is something you simply cannot do if the pattern and the provider are tightly coupled.

Lab Task 2:

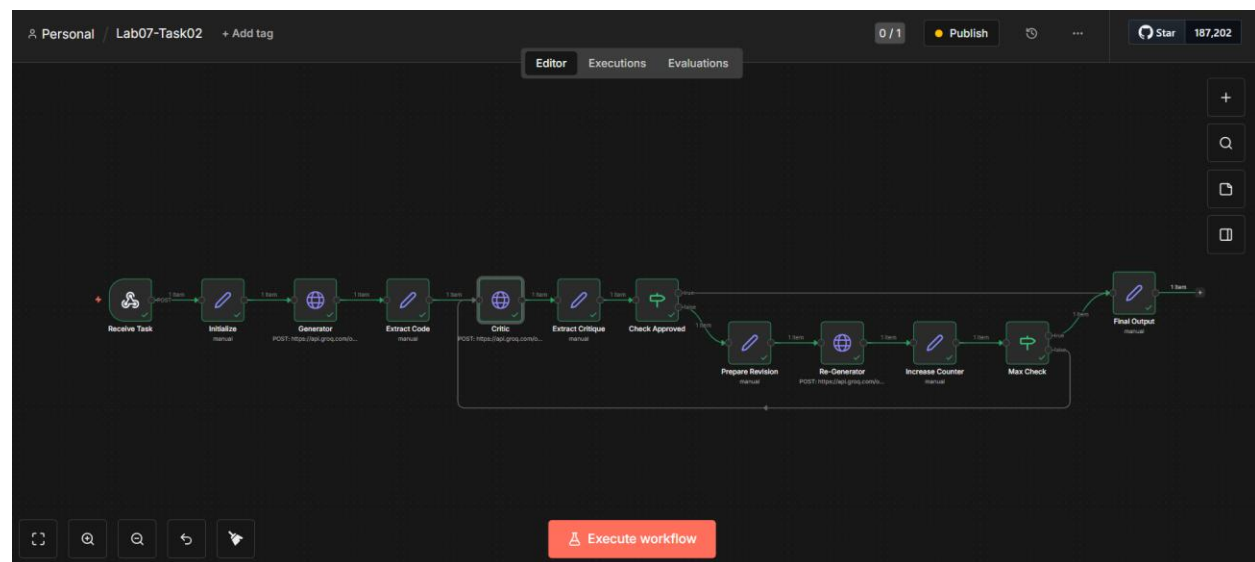
Replicate the Reflection Loop in n8n

- *Simple Task (Quick Approval):*

Query:

```
{
  "task": "Write a Python function that checks if a patient's BMI falls into
underweight, normal, overweight, or obese categories and returns an appropriate health
message."
}
```

Workflow:



Postman:

The screenshot shows a Postman interface with a POST request to `http://localhost:5678/webhook-test/reflect`. The request body is a JSON object:

```
{
  "task": "Write a Python function that checks if a patient's BMI falls into underweight, normal, overweight, or obese categories and returns an appropriate health message."
}
```

The response is a 200 OK status with a JSON body:

```
{
  "final_code": "def check_bmi(weight: float, height: float) -> str:\n    \"\"\"\n    Checks if a patient's BMI falls into underweight,\n    normal, overweight, or obese categories.\n    Args:\n        weight (float): Patient's weight in kg.\n        height (float):\n        Patient's height in meters.\n    Returns:\n        str: An appropriate health message based on the patient's BMI.\n    Raises:\n        ValueError: If weight or height is not a positive number.\n    \"\"\"\n    if weight <= 0 or height <= 0:\n        raise ValueError(\"Weight and height must be positive numbers.\")\n    bmi = weight / (height ** 2)\n    if bmi < 18.5:\n        return \"You are underweight. Please consult a doctor to discuss ways to gain weight healthily.\"\n    elif 18.5 <= bmi < 25:\n        return \"Your weight is normal. Keep up the good work!\"\n    elif 25 <= bmi < 30:\n        return \"You are overweight. Please consider a healthy diet and regular exercise to lose weight.\"\n    else:\n        return \"You are obese. Consult a doctor to discuss a weight loss plan and improve your overall health.\"",
  "round_count": 1,
  "approved": true,
  "domain": "Medical AI"
}
```

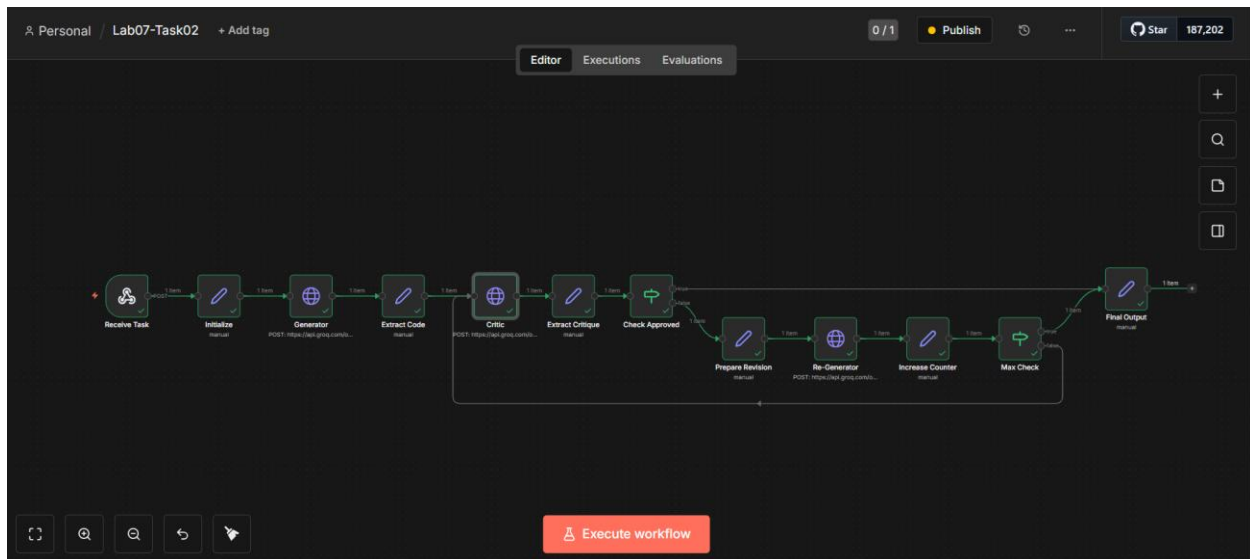
Lab 07

- *Complex Task – Multiple Rounds*

Query:

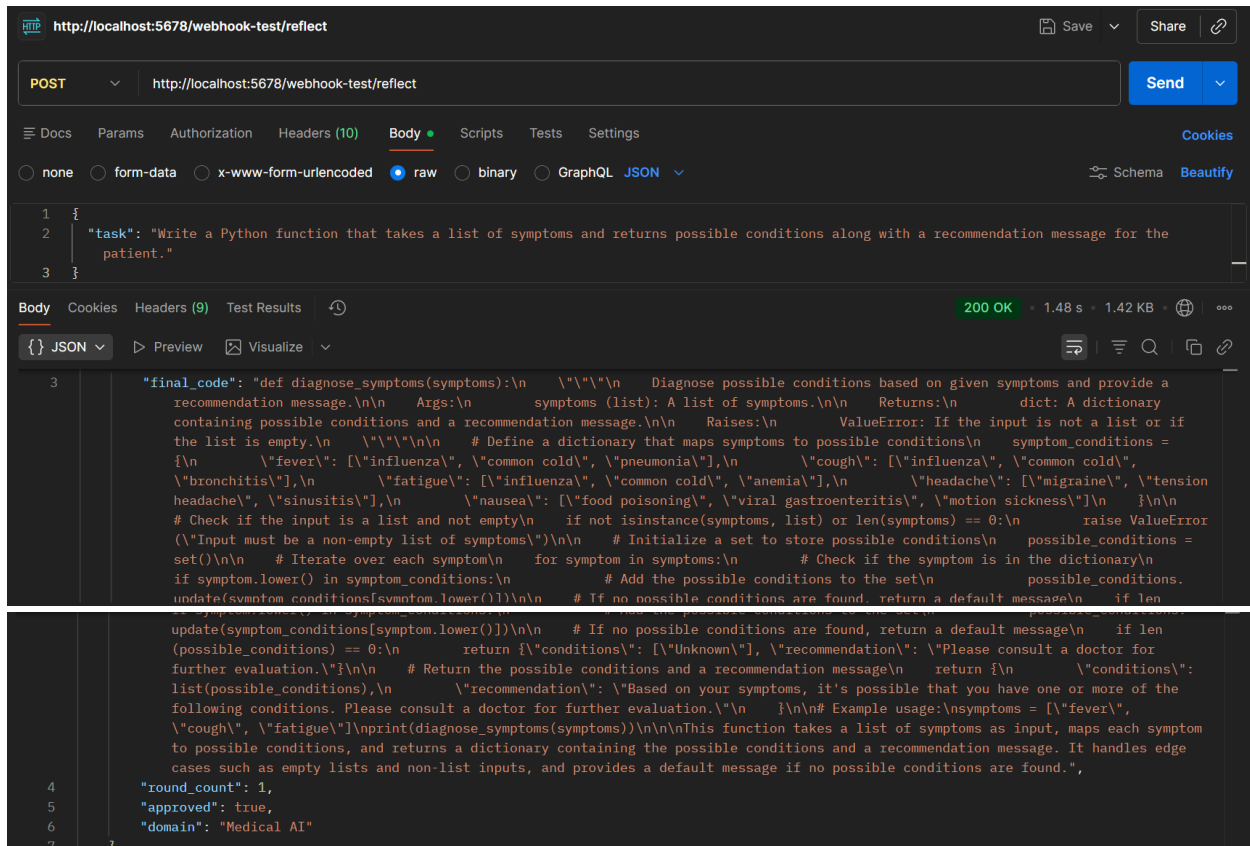
```
{  
  "task": "Write a Python function that takes a list of symptoms and returns possible  
  conditions along with a recommendation message for the patient."  
}
```

Workflow:



Lab 07

Postman:



- *Forced Early Stop*

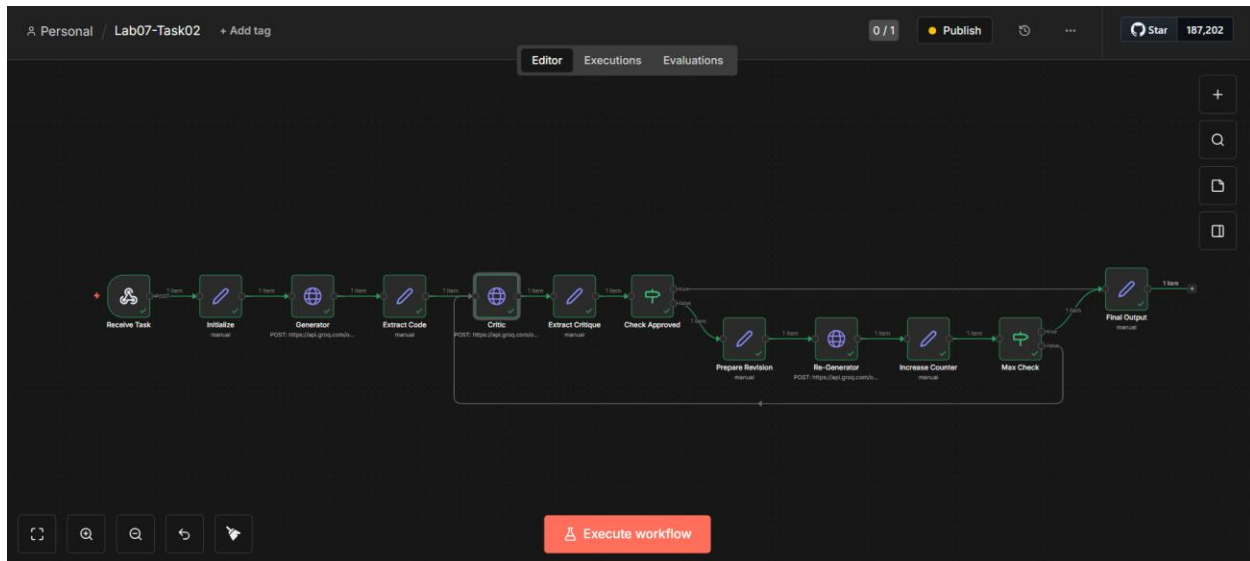
Query:

```
{
  "task": "Write a Python function that suggests medication dosage based on patient weight."
}
```

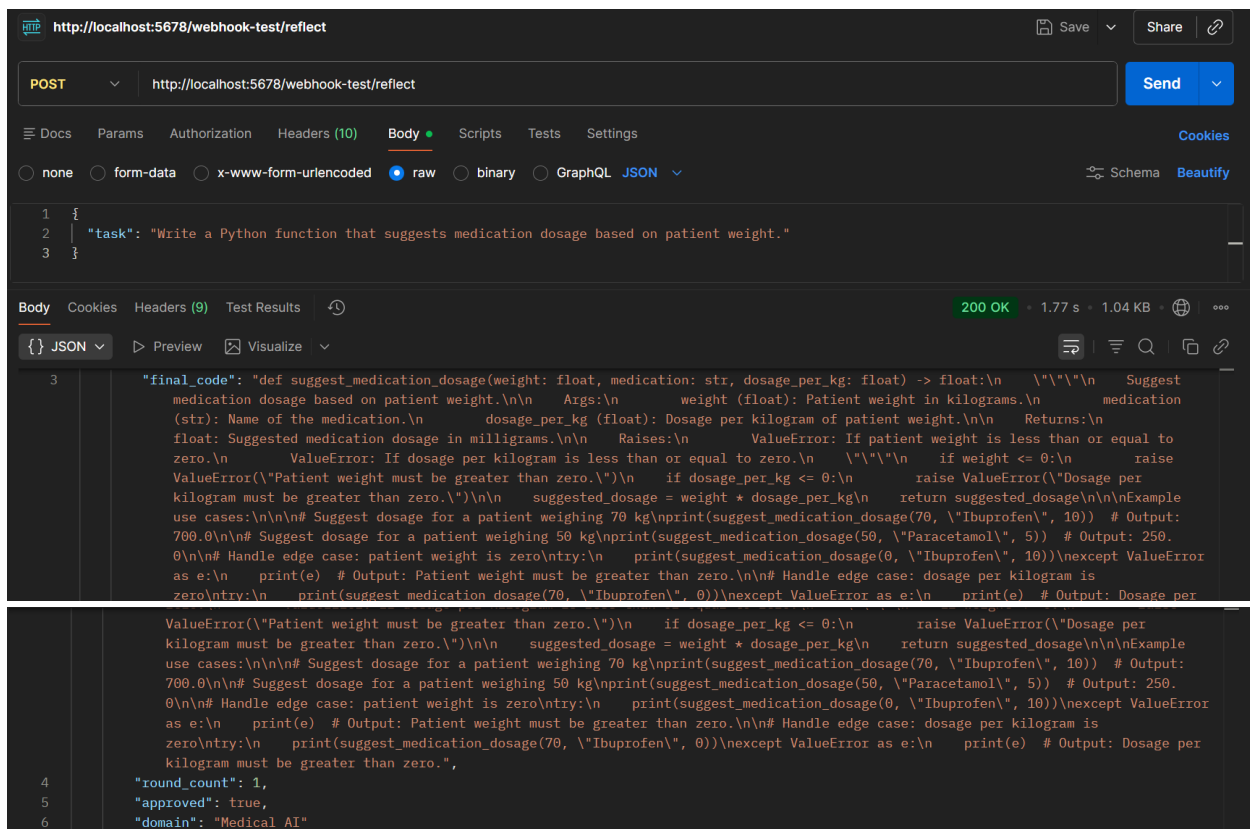
Lower-quality code compared to `max_rounds=3`.

Workflow:

Lab 07



Postman:



Reflection:

Medical AI code carries real risk because a function that outputs health advice without safety disclaimers or referral language could genuinely mislead a patient into making a dangerous decision. The five-criterion checklist forces the Critic to check for things like harmful

Lab 07

recommendations and unsupported medical claims that a standard code reviewer would never even think to look for. Without this domain-specific reflection loop, the Generator would produce code that is technically correct Python but completely unsafe to deploy in any real healthcare context.

-END-